

Video Game Pricing

By: Group 15 (Seven (Stephani) Soriano, Satyansh Rai, Karla Nguyen, Jay Barrios Abarquez)

Table of Contents

Abstract.....	1
Introduction.....	1
2.1 Background and Motivation.....	1
2.2 Problem Statement.....	2
2.3 Project Goals.....	2
Dataset Description.....	3
3.1 Data Source.....	3
3.2 Features and Variables.....	3
3.3 Data Cleaning and Preprocessing.....	4
Exploratory Data Analysis (EDA).....	5
4.1 Overview of Data Distributions.....	5
4.2 Analysis of Pricing Trends.....	5
4.3 Research Questions and Visualizations.....	6
4.4 Key Insights from EDA.....	12
Methodology.....	13
5.1 Data Preparation.....	13
5.2 Target Variable Construction.....	13
5.3 Train/Test Split.....	14
5.4 Feature Engineering and Pipeline.....	14
5.5 Model Training Strategy.....	14
5.6 Evaluation Approach.....	15
Machine Learning Models.....	15
6.1 Linear Regression.....	15
6.2 XGBoost.....	17
6.3 Random Forest Regression.....	19
6.4 Best Model Selection.....	22
Program Design and Implementation.....	22
7.1 Overall System Design.....	22
7.2 Code Structure.....	22
7.3 Key Code Snippets.....	23
Instructions for Running the Program.....	23
Conclusion.....	24
8.1 Summary of Findings.....	24
8.2 Limitations.....	24
8.3 Future Work.....	25
References.....	26

Abstract

This project takes video prices using Steam dataset information to look at important pricing factors and build Machine Learning (ML) models for both regression and classification problems. After utilizing common preprocessing tools, we removed many leakage-prone price variables and utilized features such as release year, recommendations, Metacritic information, and genres to truly understand the pricing of games and how they are determined. Exploratory analysis illustrated that most games are lower priced, with relatively few premium titles, creating both skewness and class imbalance towards lower budget, lower priced games. We compared Linear Regression, XGBoost, and Random Forest models. In the end, Random Forest performed the best out of our analysis for price prediction, achieving a test RMSE of 7.61 and R^2 of 0.290, while also producing the strongest overall classification balance with 83.7% accuracy and an F1-score of 0.276. These results show that, while working well enough on “budget” games, the skewness makes it difficult to predict “premium” level games. In essence, it concludes that video game pricing is influenced by complex relationships, not something that can be described linearly, needing more detailed features.

Introduction

2.1 Background and Motivation

The video game industry has experienced rapid growth over the past decade, with thousands of new titles released annually across multiple platforms. As the market expands, the pricing of video games has become increasingly complex, influenced by factors such as genre, release year, content availability, and overall popularity. Consumers often face difficulty determining whether a game is fairly priced, while developers and publishers must carefully balance pricing strategies to remain competitive and profitable.

With the availability of large-scale datasets such as SteamDB, it is now possible to analyze historical pricing data and uncover patterns that influence game pricing. Understanding these patterns can provide valuable insights into how different features contribute to price variation and can support more informed decision-making for both consumers and developers.

2.2 Problem Statement

Despite the abundance of available data, there is limited clarity on how specific game characteristics influence pricing. Factors such as genre, downloadable content (DLC), release year, and platform compatibility may all contribute to price differences, but their exact relationships are not always obvious.

The problem addressed in this project is to determine how various features of video games affect their pricing and to develop predictive models that can estimate a game's price or classify it into pricing categories. This involves both analyzing historical data and applying machine learning techniques to identify meaningful patterns and relationships.

2.3 Project Goals

The primary goals of this project are:

- To analyze how different features influence video game pricing
- To identify trends and patterns in pricing over time
- To build machine learning models that predict the price of a game
- To classify games into pricing categories (premium vs. budget)
- To gain insights into which factors are most significant in determining price

These goals combine exploratory data analysis with predictive modeling to better understand pricing behavior in the video game industry.

Dataset Description

3.1 Data Source

The dataset used in this project was obtained from Kaggle's Steam Dataset (2025), which makes use of SteamDB, a publicly available database that provides detailed information about games on the Steam platform. The dataset itself consists of multiple CSV files, including game metadata, pricing information, and genre classifications. These files were then strategically combined to create a usable dataset fitting for our analysis.

3.2 Features and Variables

The dataset contains a variety of features describing each game. Key variables used in this project include:

- **mat_final_price**: Current / recorded Steam price in dollars. This becomes price_target for regression and drives the $\geq \$20$ "premium" classification target.
- **mat_initial_price**: List price before discounts (dollars after conversion); used throughout EDA and discount derivations; excluded from X as leakage-prone alongside the final price.
- **mat_discount_percent**: Store discount percentage where present; listed among leakage-prone fields and dropped from the supervised feature matrix.
- **discount_amount**: Derived as initial minus final price in notebooks (e.g. discount-level plots); not used as an input feature in X for the same leakage reason.
- **is_free**: Free-to-play flag; central to coloring / interpretation in plots; excluded from X with other price-leakage columns.
- **metacritic_score**: Metacritic aggregate score when available; used as a numeric predictor in X and in pricing / review plots.

- **recommendations_total**: Total Steam user recommendations as a popularity / engagement proxy; in X and in scatter / discount analyses.
- **has_metacritic**: Binary flag that a Metacritic score exists; included in X as in X_test.csv.
- **release_year**: Release calendar year (from release_date or a native release_year column); required context feature in build_feature_matrix.
- **name**: Game title; kept for tables and hover text, dropped before building the numeric feature matrix.
- **Genre indicators**: One-hot columns Action, Adventure, Casual, Free to Play, Indie, Massively Multiplayer, Other, RPG, Racing, Simulation, Sports, Strategy, Unknown—explicitly required and appended in train_test_split_setup.py for X.
- **price_category**: Binned price labels used in categorical / box plots; treated like other direct price encodings and not fed into X.

For the purposes of machine learning, features that directly reveal or strongly correlate with the target variable (such as final price, discount-related values, and free indicators) were removed to prevent data leakage.

3.3 Data Cleaning and Preprocessing

Before analysis and modeling, the dataset underwent several preprocessing steps to ensure quality and consistency. First, game metadata was merged with genre information using a common identifier (appid) to create a unified dataset. Irrelevant features, including identifier columns, attributes with a lot of text, and non-informative metadata, were removed to focus on variables suitable for machine learning. Missing values in numerical features were handled using median imputation, while our binary feature has_metacritic was introduced to preserve information about whether a game had an associated review score.

Price-related features were processed by converting values from cents to dollars and removing games with initial or final prices exceeding \$100 to reduce the impact of outliers. Furthermore, the release date was transformed into a numerical `release_year` feature to capture temporal trends in pricing. In terms of categorical genre information, we converted various genres into one-hot encoded features, allowing them to be usable by our models. These preprocessing steps ensured that the dataset was clean, consistent, and suitable for both exploratory analysis and machine learning tasks.

Exploratory Data Analysis (EDA)

4.1 Overview of Data Distributions

Exploratory Data Analysis was conducted to examine the structure of the Steam dataset and understand pricing behavior. Prices were converted from cents to U.S. dollars, and free-to-play games were assigned a value of \$0.

Histograms of initial and final prices (capped at \$100) show a strong right-skewed distribution. Most games are priced at the lower end, including many free titles, while relatively few fall into higher price ranges. This indicates that low-cost games dominate the platform, with premium titles forming a smaller segment.

4.2 Analysis of Pricing Trends

Pricing trends were analyzed over time and across key features. Median and mean final prices were plotted by release year to observe how typical pricing changes over time.

Relationships between price and other variables were explored using scatterplots. Initial price was compared with total recommendations, with distinctions made between free and paid games. Faceted plots by price category (Free, Low, Medium, High) showed that this relationship varies across pricing tiers.

Additional analysis included:

- Pairwise comparisons between price, recommendations, and Metacritic score

- Boxplots of Metacritic scores across final price categories
- Discount analysis based on the difference between initial and final price
- Scatterplots examining potential predictors of final price

These visualizations highlight that pricing is influenced by multiple interacting factors rather than a single variable.

4.3 Research Questions and Visualizations

To guide the analysis, several research questions were explored using visualizations:

1. How are initial and final Steam prices distributed?

Histograms were used to examine the distribution of prices after conversion to U.S. dollars, highlighting the concentration of games in lower price ranges.

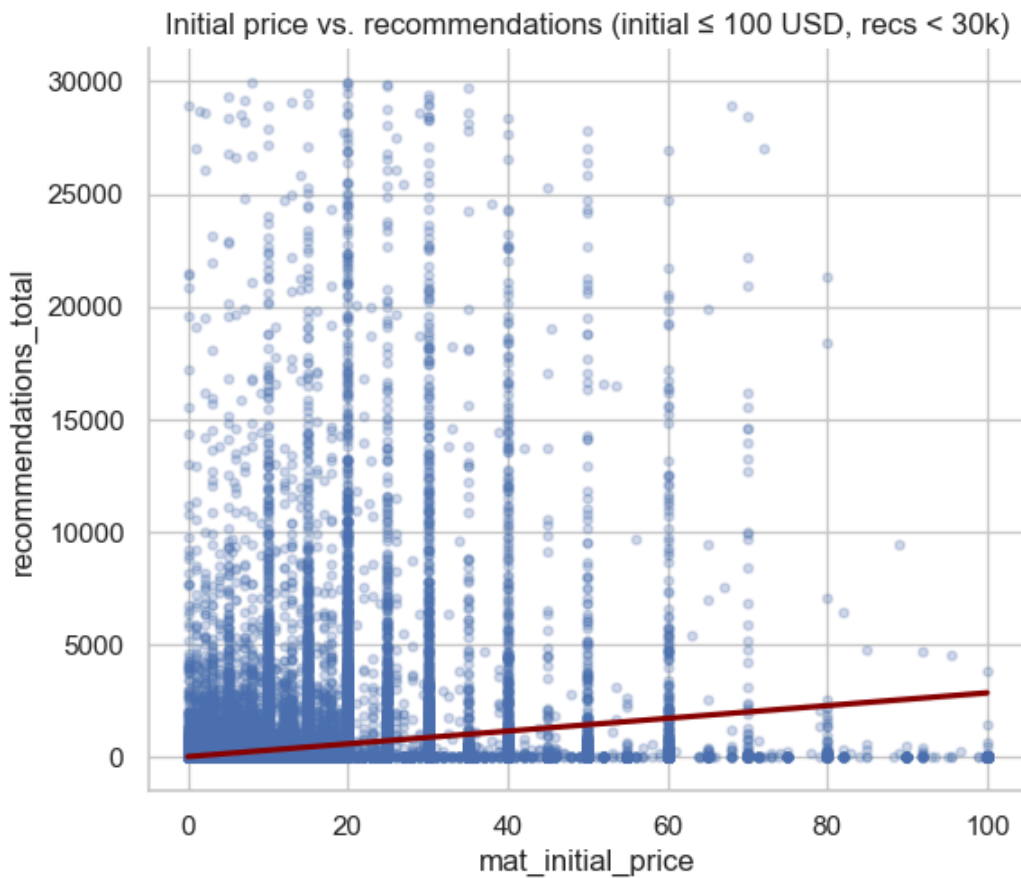
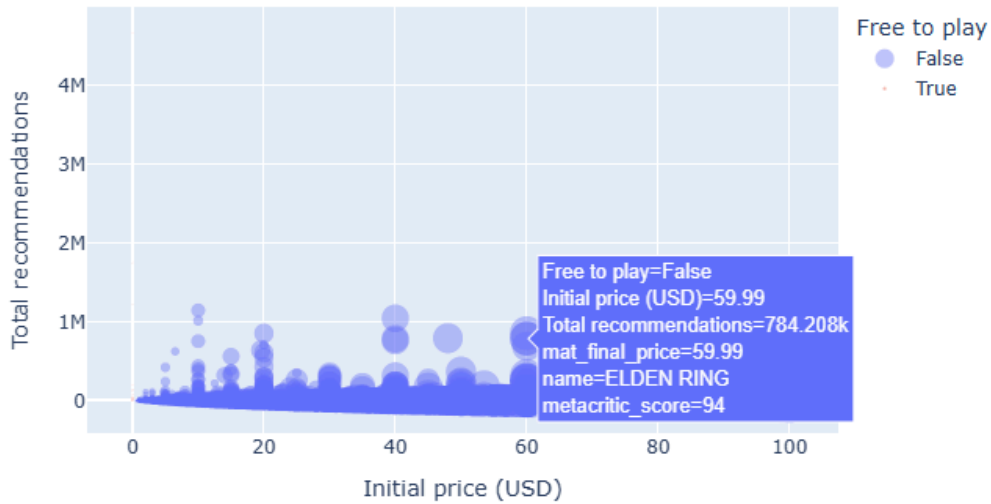


2. How do typical final prices change over time?

Line plots of median and mean prices by release year were used to analyze pricing trends across different time periods.

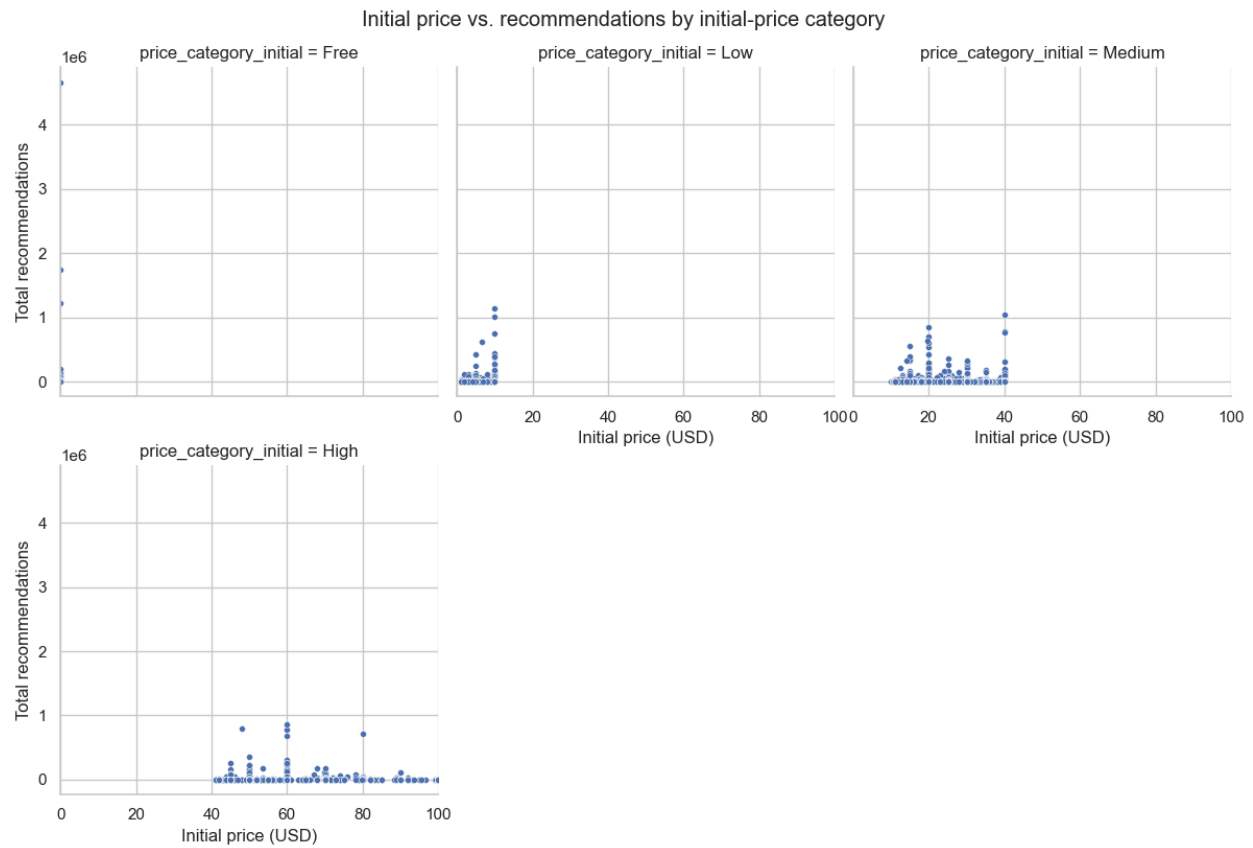


Interactive: initial price vs. recommendations (hover / zoom / pan)



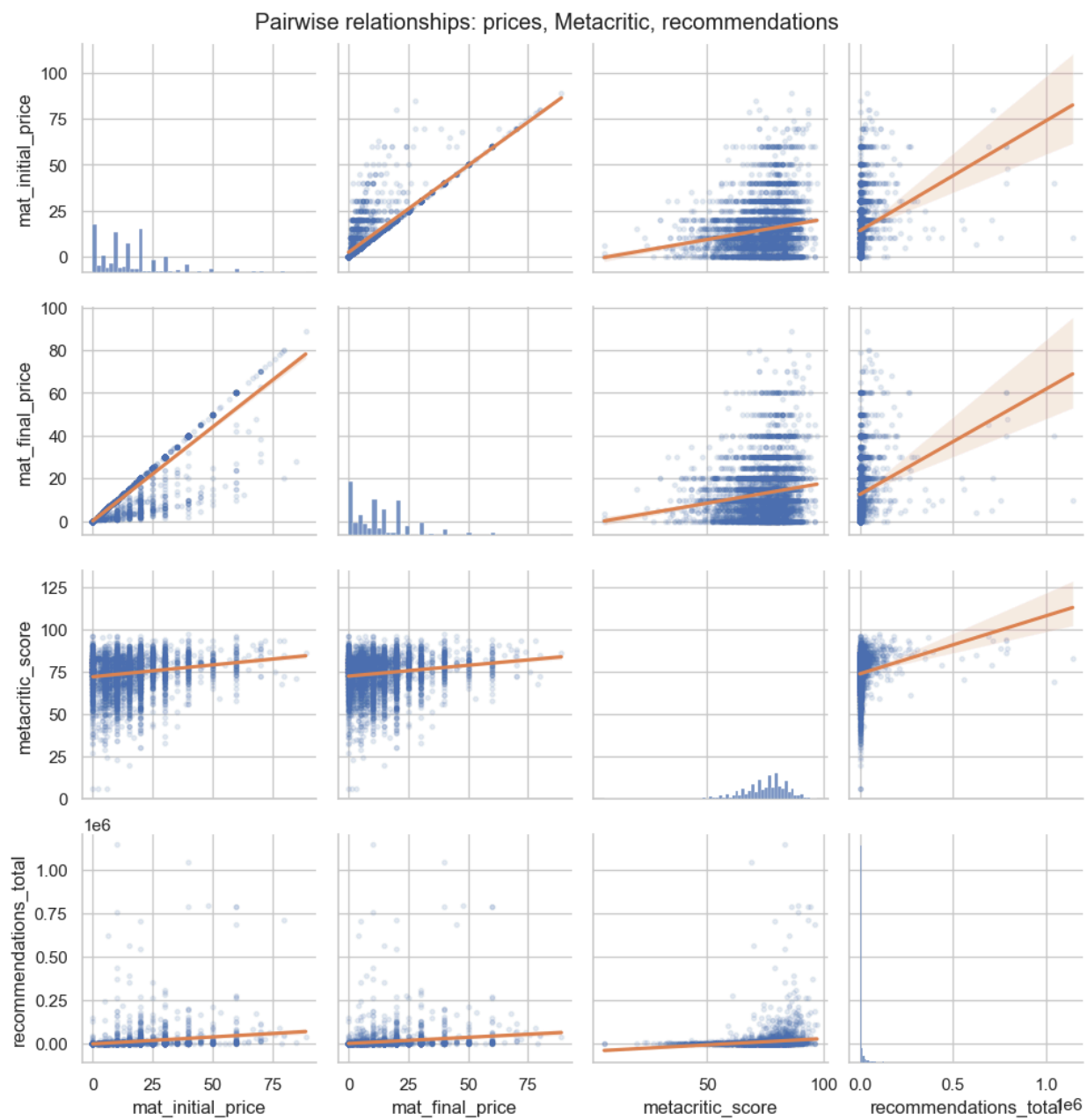
4. How does this relationship vary across price categories?

Faceted scatterplots compared patterns within Free, Low, Medium, and High price groups.



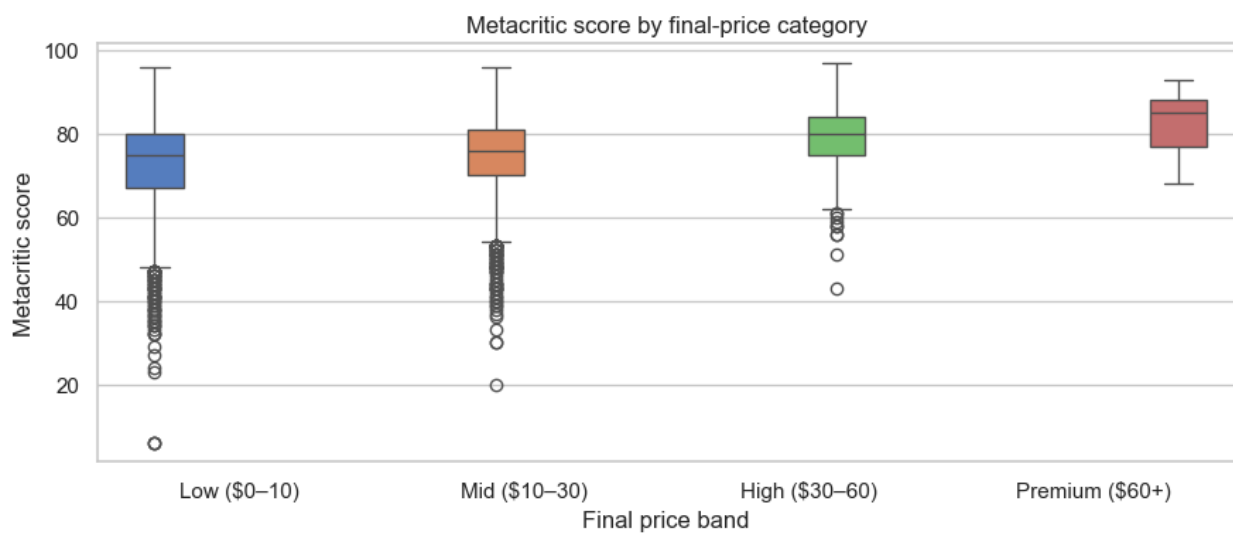
5. How do price, Metacritic score, and recommendations vary together?

A pairplot was used to examine pairwise relationships among key numeric variables.



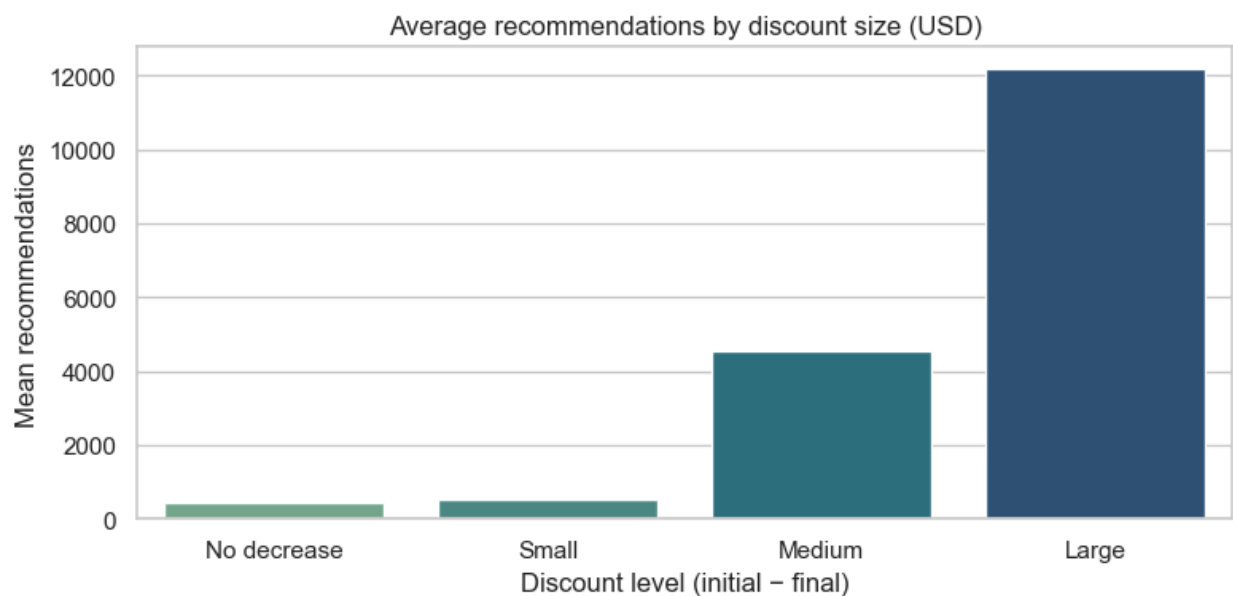
6. **Do Metacritic score distributions differ across price tiers?**

Boxplots compared review scores across different final-price categories.



7. **Does discount size relate to average recommendations?**

Bar charts compared mean recommendation counts across discount levels.



8. Which variables appear most associated with final price?

Scatterplots examined the relationship between final price and potential predictors such as initial price, recommendations, and Metacritic score.



These visualizations collectively provided insight into pricing behavior and the relationships between price and other game attributes.

4.4 Key Insights from EDA

The EDA reveals several key patterns:

- Game prices are highly right-skewed, with most games priced low or free
- Pricing trends vary over release years
- Price is related to popularity and review scores, but not in a simple linear way
- Discounting appears to be associated with differences in player engagement
- Multiple factors—including initial price, recommendations, review scores, and release timing—interact to influence final price

Overall, these findings suggest that pricing behavior is complex and requires models capable of capturing nonlinear relationships and feature interactions.

Methodology

5.1 Data Preparation

The Steam dataset used in this project was first preprocessed to ensure it was suitable for machine learning tasks. Since the goal was to predict game prices and classify games into pricing categories, it was important to carefully select features that would not introduce data leakage. Columns directly related to pricing outcomes such as final price, initial price, discount percentage, discount amount, and indicators like whether a game is free were removed from the feature set. These variables either directly determine the target variable or are derived from it, and including them would lead to artificially inflated model performance.

Only numerical and boolean features were retained for modeling. Boolean values were converted into integer format to ensure compatibility across all machine learning models. Missing values in numerical columns were handled using median imputation based on the training data to avoid bias from extreme values.

5.2 Target Variable Construction

Two target variables were created to support both regression and classification tasks:

- Regression Target: The continuous variable representing the final price of a game.
- Classification Target: A binary variable indicating whether a game is considered “premium” or “budget.”

To construct the classification target, a threshold of \$20 was used. Games with a final price greater than or equal to this threshold were labeled as “premium,” while those below were labeled as “budget.” This threshold was selected as a reasonable cutoff to distinguish between lower-cost and higher-cost games on the platform.

5.3 Train/Test Split

The dataset was divided into training and testing sets using an 80/20 split. The training set was used to train the machine learning models, while the testing set was reserved for evaluating final model performance on unseen data. To ensure consistency across both regression and classification tasks, a single split was applied to all features and target variables. Additionally, stratified sampling was used based on the classification target to preserve the proportion of premium and budget games in both the training and testing sets. This helps prevent class imbalance from affecting model evaluation.

A fixed random seed was used during the split process to ensure reproducibility of results.

5.4 Feature Engineering and Pipeline

Feature preprocessing was designed to standardize the data and prepare it for modeling. Numerical features were scaled where necessary, and categorical variables were transformed into numerical representations using encoding techniques. To maintain consistency across models, preprocessing steps were designed to be reusable and applied uniformly to both training and testing data. This ensures that all models are trained and evaluated under the same conditions. In addition, we wanted to avoid data leakage by ensuring that preprocessing steps such as imputation were fitted only on the training data and then applied to the test data.

5.5 Model Training Strategy

Multiple machine learning models were trained to address both regression and classification objectives. For regression, models were used to predict the price of a game, while classification models were used to determine whether a game falls into the premium or budget category.

To evaluate model performance, **k-fold cross-validation** was applied to the training data. This approach involves dividing the training data into multiple folds and training the model on different subsets while

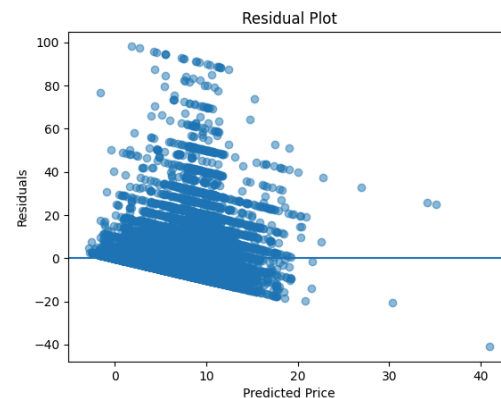
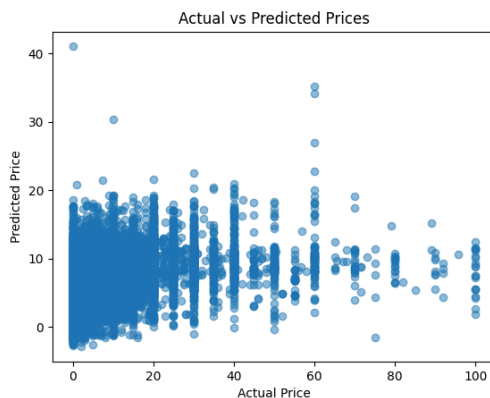
validating on the remaining data. This provides a more reliable estimate of model performance compared to a single train/test split. All models were trained using the same training data and evaluated using consistent metrics to allow for fair comparison.

5.6 Evaluation Approach

After training and cross-validation, the best-performing models were selected based on their evaluation metrics. These models were then tested on the held-out test set to assess their performance on unseen data. For regression models, performance was evaluated using metrics such as Mean Squared Error (MSE) and R^2 score. For classification models, evaluation metrics included accuracy, precision, recall, and F1-score. A confusion matrix was also used to provide a detailed view of classification performance. This structured evaluation approach ensures that model performance is both reliable and generalizable.

Machine Learning Models

6.1 Linear Regression



```
Cross-validation R^2 scores: [0.14159071 0.1110348 0.13512846 0.12927656 0.13460723]
Average CV R^2: 0.13032755199239704
WITHOUT SCALING
MSE: 69.87917930557428
R^2: 0.14408241418502643

Scaled Cross-validation R^2 scores: [0.14159071 0.1110348 0.13512846 0.12927656 0.13460723]
Scaled Average CV R^2: 0.13032755199238816

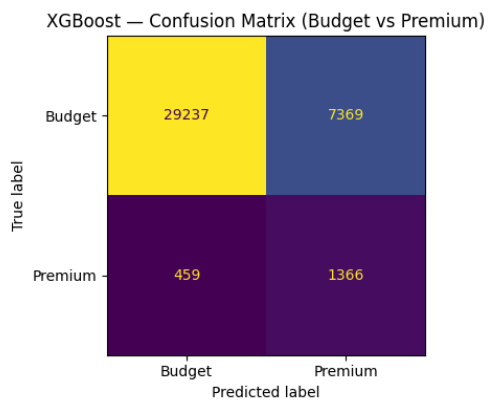
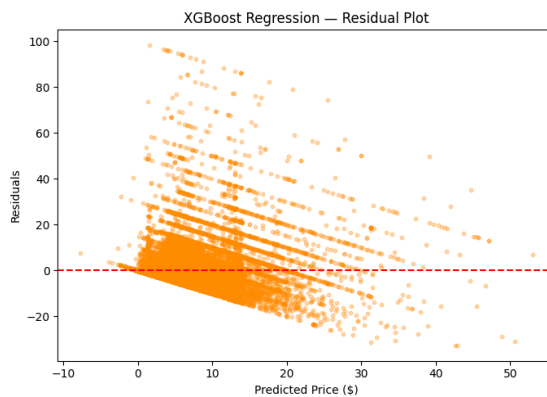
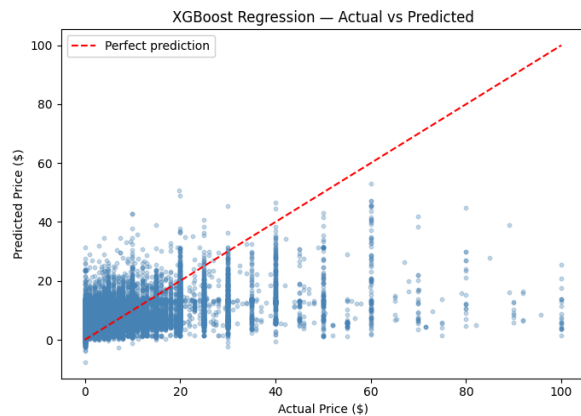
WITH SCALING
MSE: 69.87917930557457
R^2: 0.14408241418502288
PS C:\CompSci\CS 133\term-project-group-15> █
```

Linear regression was implemented as a baseline model to predict game prices using numerical and encoded features. Both scaled and unscaled versions were tested, and 5-fold cross-validation was applied to evaluate model stability. Scaling did not significantly impact performance, which is expected for linear regression.

In terms of model performance, we evaluated Linear Regression using Mean Squared Error (MSE) and R^2 . The model achieved an R^2 of approximately 0.14 and an MSE of about 70, indicating a limited ability to accurately predict game prices. 5-fold cross-validation produced an average R^2 of approximately 0.13, with consistent results across folds. This suggests that the model's performance is stable but consistently low, indicating poor generalization. Feature scaling did not significantly impact performance since both scaled and unscaled models produced nearly identical results.

In the context of this project, these results imply that game pricing is influenced by complex, non-linear factors that are not well represented by a simple linear model. The model tends to predict prices within a narrow range, making it less effective at capturing higher-priced or more variable games.

6.2 XGBoost



===== SAMPLE PREDICTIONS (first 10 test rows) =====

Actual Price (\$)	XGB Predicted (\$)	Error (\$)	Actual Label	XGB Label	Pred
0.00	0.23	-0.23	Budget	Budget	
4.99	9.75	-4.76	Budget	Premium	
0.00	1.39	-1.39	Budget	Budget	
9.99	5.18	4.81	Budget	Budget	
5.99	5.37	0.62	Budget	Budget	
14.99	5.10	9.89	Budget	Budget	
39.99	6.69	33.30	Premium	Budget	
10.99	13.07	-2.08	Budget	Premium	
0.00	2.53	-2.53	Budget	Budget	
1.99	5.75	-3.76	Budget	Budget	

XGBoost (Extreme Gradient Boosting) was selected as an advanced model capable of capturing complex, non-linear relationships in the data. Unlike Linear Regression, which assumes a linear relationship between feature and price, XGBoost builds decision trees sequentially, with each tree correcting the errors of previous ones. This allows the model to learn interactions between features such as genre, release year,

and popularity metrics. Additionally, XGBoost provides feature importance scores, which help identify which factors have the greatest influence on pricing.

For the regression task, XGBoost was used to predict the final price of a game. We evaluated the model with MSE, RSME, and R^2 .

Using 5-fold cross-validation, the model achieved:

- Average R^2 : 0.269
- Average RMSE: 7.67

On the test set:

- MSE: 58.95
- RMSE: 7.68
- R^2 : 0.278

This means that, on average, the model's predictions are off by about \$7.68, and it explains approximately 27.8% of variance in game prices.

For the classification, XGBoost was used to predict whether a game is "budget" or "premium." Since the dataset is highly imbalanced, with many more budget games than premium ones, we used the `scale_pos_weight` parameter to adjust for class imbalance. Model was evaluated using accuracy, precision, recall, and F1-score:

Cross-validation results:

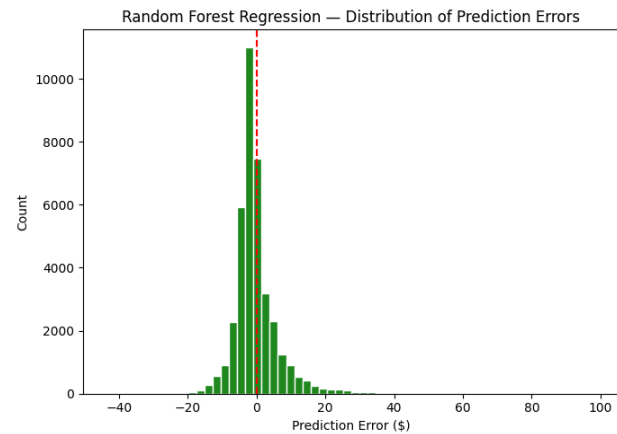
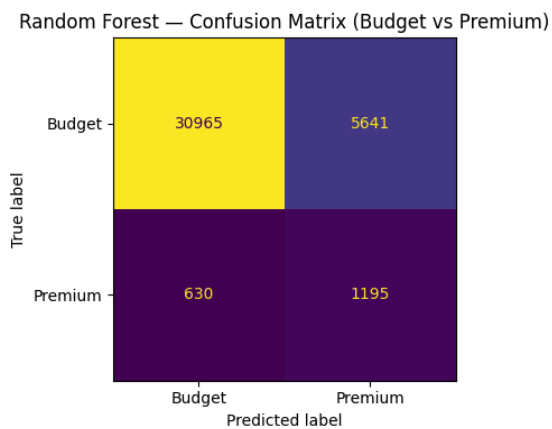
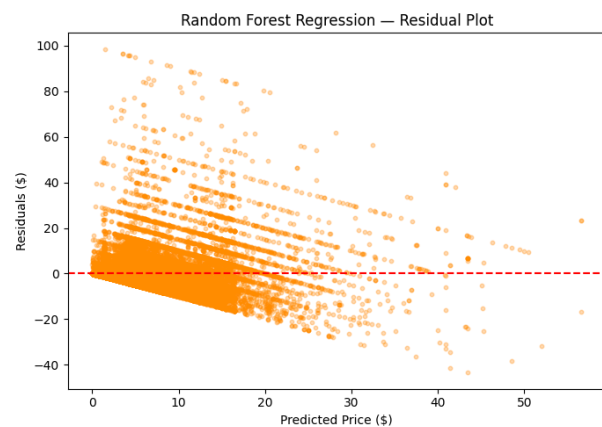
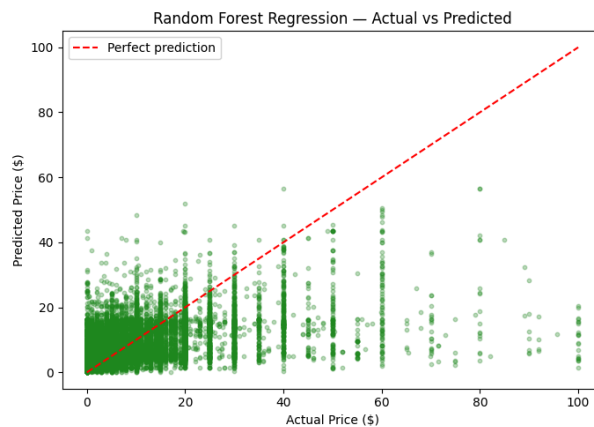
- Accuracy: 0.799
- F1-score: 0.261

Test set results:

- Accuracy: 0.796 (79.6%)
- Precision: 0.156 (15.6%)
- Recall: 0.749 (74.9%)
- F1-score: 0.259

These results show that while the model has high accuracy, this is influenced by class imbalance. The relatively low precision indicated that many predicted premium games are actually budget games. However, the high recall suggests the model is effective at identifying most premium games, which may be more important depending on the application.

6.3 Random Forest Regression



===== SAMPLE PREDICTIONS (first 10 test rows) =====						
Actual Price (\$)	RF Predicted (\$)	Error (\$)	Actual	Label	RF Label	Pred
0.00	0.16	-0.16	Budget			Budget
4.99	8.96	-3.97	Budget			Premium
0.00	1.38	-1.38	Budget			Budget
9.99	5.26	4.73	Budget			Budget
5.99	6.87	-0.88	Budget			Premium
14.99	5.48	9.51	Budget			Budget
39.99	10.87	29.12	Premium			Budget
10.99	16.30	-5.31	Budget			Premium
0.00	2.71	-2.71	Budget			Budget
1.99	5.83	-3.84	Budget			Budget

Random Forest was used to test out a more efficient model to predict game prices. While Linear Regression worked, it had some inconsistencies and inaccurate reporting. Random Forest Regression is a method that predicts continuous values by processing the output of a multitude of decision trees. Every tree is essentially trained on a random sample of data with a random subset of features. The final answer then becomes a culmination of the averages of all tree predictions. This means utilizing multiple features in the prediction as a whole. This makes Random Forest a good choice because our dataset had a lot of features that would help in the prediction determination. We used several hyperparameters to determine the best model. The best performing one used 400 trees, a maximum depth of 20, a minimum split size of 10, a minimum leaf size of 1, and square-root feature sampling.

We split this into two solutions, a regression analysis and a classification analysis. We first utilized Random Forest Regression using MSE, RMSE, and R^2 metrics.

Using 5-fold cross-validation, the model achieved:

- Average R^2 : 0.280
- Average RMSE: 7.61

On the test set, the model achieved:

- MSE: 57.95
- RMSE: 7.61

- R^2 : 0.290

To interpret the results, this means that, on average, the model's predictions are off by about \$7.61, and it explains approximately 29.0% of variance in game prices. Essentially meaning the prices predicted can sometimes go much further off than the actual prices. The sample data shows this well where the RF predicted for one of them was \$10.87 where the actual should have been \$39.99.

Similar to XGBoost, we used the same metrics to solve the same categorization problem. Below are the results of this:

Cross-validation results:

- Accuracy: 0.839
- F1-score: 0.278

Test set results:

- Accuracy: 0.837 (83.7%)
- Precision: 0.175 (17.5%)
- Recall: 0.655 (65.5%)
- F1-score: 0.276

These results show promising results in terms of accuracy but the data is still quite imbalanced. The low precision shows that many games that should have been described as premium were labeled as budget. Nonetheless, the recall is still shown to be fairly high being 65.5% which can showcase that the model is still capable of higher results, but still comes up short in some classification. Compared to the Linear Regression model, the Random Forest model had much higher metrics. This means that Random Forest was slightly more balanced overall. As shown in the sample output above, the Random Forest Classification was decent enough at determining budget games but unfortunately came up short when classifying premium games.

6.4 Best Model Selection

After conclusively looking at the different possible models that can be used, it appears that Random Forest yielded the best results. Compared with the XGBoost classification model (accuracy = 0.796, precision = 0.156, recall = 0.749, F1 = 0.259), the Random Forest classifier had higher accuracy, precision, and F1-score, but a lower recall. This means Random Forest was slightly more balanced, but XGBoost was better at finding a larger share of premium games. While this shows as a better solution to the questions we defined above, it is still not as well defined as we had hoped due to the data being slightly imbalanced. All in all, even though our models had some limitations to them, Random Forest was still the best choice that we tested by far.

Program Design and Implementation

7.1 Overall System Design

The system was designed as a modular machine learning pipeline that processes raw data, trains models, and evaluates performance. The workflow begins with data preprocessing, where features are cleaned, transformed, and split into training and testing sets. These processed datasets are then used to train multiple machine learning models, including Linear Regression, Random Forest, and XGBoost.

Each model follows the same pipeline to ensure consistency and fairness in evaluation. Cross-validation is applied during training to estimate model performance, and the final evaluation is conducted on a held-out test set. The system also includes visualization components to display model performances and insights, such as prediction plots and confusion matrices.

7.2 Code Structure

The program is organized to sections:

- Data Loading: Reads preprocessed training and test dataset from CSV files
- Model Training: Implements regression and classification models

- Cross-Validation: Evaluates models using k-fold validation
- Testing: Generates predictions on unseen test data
- Evaluation: Computes metrics such as MSE, R^2 , accuracy, and F1-score
- Visualization: Produces plots for analysis (scatter plots, confusion matrix, feature importance)

7.3 Key Code Snippets

Instructions for Running the Program

1. Ensure all required Python libraries are installed:

```
Python  
pip install pandas numpy matplotlib scikit-learn xgboost
```

2. Place the dataset files in the correct directory: **Project Assignments/models/ml_splits/**
3. Ensure the dataset files are present:
 - X_train.csv
 - X_test.csv
 - y_reg_train.csv
 - y_reg_test.csv
 - y_cls_train.csv
 - y_cls_test.csv
4. Run the Python scripts

```
Python  
python lin_reg.py  
  
python random_forrest_regression.py  
  
python xgboost.py
```

Conclusion

8.1 Summary of Findings

When exploring the data and analyzing the trends, we noticed how many games there were in the lower price bracket. This made the data heavily skewed for lower priced games, whereas, the higher priced games would be less, possibly explained by higher development costs and time. With this analysis we went into the model creation knowing that there would be a skewness towards “budget games”. After forming thoughtful questions, exploring the dataset, and testing out multiple models, we came to the conclusion that, from our testing, Random Forest would be the best choice of model. While the dataset had its own limitations as described, it still showed the prices of budget games accurately and was level with our initial hypothesis of how the model would predict the data. The model still did a decent job of predicting “budget” games and their prices, while when it came to “premium” games, struggled to get the desired results we wanted.

8.2 Limitations

Several limitations affected the models' performance. First, the dataset exhibited strong class imbalance, which impacted classification precision. Second, the available data and features were limited and did not fully capture important aspects of game value. While variables such as recommendations and Metacritic scores were included, other influential factors such as the number of DLCs, post-launch content, and developer or publisher characteristics were not available. In particular, missing information about developer reputation (e.g., popularity, past success, indie vs. large studio) and content depth (such as DLC volume or updates) likely reduced the model's ability to accurately capture pricing differences.

Finally, the relatively low R^2 score suggests that game pricing is influenced by complex factors that are not fully represented in the dataset, limiting the overall predictive performance.

8.3 Future Work

Future improvements could focus on enhancing both the dataset and modeling approach:

- Applying hyperparameter tuning (e.g., GridSearchCV) to further optimize model performance
- Incorporating additional features such as:
 - DLC count and post-launch content
 - Developer and publisher statistics (e.g., popularity, historical success, indie vs. corporate studios)
 - Player engagement trends beyond total recommendations
- Exploring more advanced models, such as neural networks, to better capture nonlinear relationships
- Addressing class imbalance using resampling techniques

By expanding the feature set and refining modeling techniques, future work can better capture the complexity of video game pricing and improve prediction accuracy.

References

Articles:

<https://www.kaggle.com/code/jruots/forecasting-video-game-sales>

<https://medium.com/top-python-libraries/predicting-video-game-sales-a-comprehensive-data-analysis-and-machine-learning-workflow-a2bff85518ae>

Datasets:

<https://www.kaggle.com/datasets/abhishekgupta56447/steam-games-dataset-2025/data>

<https://steamdb.info>